

WorkflowScript: D1 – Design Specification

CSE341 — Concepts of Programming Languages

Part 1 Submission — 8 May 2026

Ömer Faruk Koç

210104004061

Gebze Technical University

May, 2026

Contents

1	Language Overview	3
1.1	Abstract	3
1.2	Design Priorities	3
1.3	Sample Program	3
2	Lexical Structure	4
2.1	Token Categories	4
2.2	Keywords	4
2.3	Operators	4
2.4	Separators	4
2.5	Comments	4
3	Syntax	5
3.1	Complete Grammar	5
3.2	Ambiguity Notes	6
3.2.1	Dangling Else	6
3.2.2	Operator Precedence	6
3.2.3	Field Access on Task-Field Keywords	6
3.2.4	Expression Statement Restriction	7
3.2.5	Assignment vs. Expression	7
4	Names, Binding, Scope, and Lifetime	7
4.1	Legal Identifiers	7
4.2	Bindings: Compile Time vs. Run Time	7
4.3	Scoping	7
4.3.1	Why Static Scoping?	8
4.3.2	What Would Break Under Dynamic Scoping?	8
4.4	Lifetime	8
4.4.1	Why Static Lifetime for Tasks?	8
4.4.2	Why Stack-Dynamic for Local Variables?	9

1 Language Overview

1.1 Abstract

WorkflowScript is a domain-specific language for defining, composing, and executing task-based workflow pipelines. Its target users are backend developers who need to model CI/CD pipelines, job scheduling, and multi-step automation without embedding that logic in a general-purpose language. The language prioritizes reliability through strong static typing, static scoping, and mandatory task-field declarations, while maintaining writability with concise syntax for common workflow patterns.

YAML-based pipeline definitions (GitHub Actions, GitLab CI) are untyped and unstructured: a misspelled field or a wrong value type produces a runtime failure, not a definition-time error. General-purpose languages add boilerplate and require recompilation for every change. WorkflowScript occupies the gap: it has enough of a programming model (functions, variables, control flow) to be expressive, while its type system and fixed task structure catch errors before any execution begins.

1.2 Design Priorities

WorkflowScript prioritizes the following Sebesta criteria:

- **Reliability** — strong static typing, static scoping, and mandatory task fields (`timeout`, `retries`, `parallel`) prevent undefined behavior at pipeline definition time. A program that passes the parser is guaranteed to have well-formed task declarations; a program that passes the type checker is guaranteed to reference only declared tasks and well-typed variables.
- **Writability** — concise syntax for the common case: a three-field `task` declaration, a one-keyword `stage` reference, and a single `run` statement to trigger a pipeline or task.
- **Readability** — structure mirrors the mental model of a workflow: tasks declared first, pipeline composition second, imperative control logic last. The keyword `on_failure` reads like plain English.

Knowingly sacrificed: **cost** (no string type, no arrays, no dynamic dispatch, no generics). These features are not needed in the pipeline domain and would complicate the type checker and interpreter without enabling any new workflow patterns.

1.3 Sample Program

```
task build {
  timeout: 30
  retries: 0
  parallel: false
}

task test {
  timeout: 60
  retries: 2
  parallel: true
}
```

Listing 1: Sample WorkflowScript program

2 Lexical Structure

2.1 Token Categories

Category	Pattern	Examples
IDENT	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>build, my_task, ci</code>
INT_LIT	<code>[0-9]+</code>	<code>0, 30, 120</code>
FLOAT_LIT	<code>[0-9]+ "." [0-9]+</code>	<code>0.5, 3.14, 60.0</code>
BOOL_LIT	<code>"true" "false"</code>	<code>true, false</code>
KEYWORD	Reserved identifiers (see below)	<code>task, if, while</code>
OPERATOR	Fixed character sequences	<code>+, ==, -></code>
SEPARATOR	Fixed single characters	<code>(, {, :</code>
COMMENT	<code>"//" .* "\n"</code>	<code>// build the service</code>

Table 1: Token categories in WorkflowScript

2.2 Keywords

The following identifiers are reserved and cannot be used as user-defined names:

<code>task</code>	<code>pipeline</code>	<code>stage</code>	<code>on_failure</code>
<code>fun</code>	<code>var</code>	<code>run</code>	<code>return</code>
<code>if</code>	<code>else</code>	<code>while</code>	
<code>int</code>	<code>float</code>	<code>bool</code>	<code>void</code>
<code>timeout</code>	<code>retries</code>	<code>parallel</code>	
<code>true</code>	<code>false</code>		

`true` and `false` are lexed as `BOOL_LIT` tokens rather than generic keywords, which simplifies the parser: they can appear anywhere a literal is expected without a separate production rule. `task` is reused as both a declaration keyword and a type name in function parameter lists. `timeout`, `retries`, and `parallel` are reserved so they cannot shadow task field names in expressions like `t.timeout`.

2.3 Operators

<code>+</code>	<code>-</code>	<code>*</code>	<code>/</code>		
<code>==</code>	<code>!=</code>	<code><</code>	<code><=</code>	<code>></code>	<code>>=</code>
<code>&&</code>	<code> </code>	<code>!</code>			
<code>=</code>	<code>-></code>	<code>.</code>			

2.4 Separators

`(` `)` `{` `}` `:` `,`

2.5 Comments

Line comments begin with `//` and extend to the end of the line. There are no block comments. The lexer discards comments entirely; they produce no tokens. Whitespace (spaces, tabs, carriage returns, newlines) is also ignored between tokens.

3 Syntax

3.1 Complete Grammar

The complete grammar in EBNF is given below.

```
<program>          ::= { <top_decl> }

<top_decl>          ::= <task_decl>
                       | <pipeline_decl>
                       | <fun_decl>
                       | <stmt>

<task_decl>         ::= "task" IDENT "{"
                       "timeout" ":" INT_LIT
                       "retries" ":" INT_LIT
                       "parallel" ":" BOOL_LIT
                       "}"

<pipeline_decl>     ::= "pipeline" IDENT "{" <pipeline_body> "}"
<pipeline_body>     ::= <stage_stmt> { <stage_stmt> } [ <on_failure> ]
<stage_stmt>        ::= "stage" IDENT
<on_failure>        ::= "on_failure" "{" { <stmt> } "}"

<fun_decl>          ::= "fun" IDENT "(" [ <params> ] ")" "->" <type> <block>
<params>            ::= <param> { "," <param> }
<param>             ::= IDENT ":" <type>

<type>              ::= "int" | "float" | "bool" | "void" | "task"

<block>             ::= "{" { <stmt> } "}"

<stmt>              ::= <var_decl>
                       | <assign_stmt>
                       | <if_stmt>
                       | <while_stmt>
                       | <run_stmt>
                       | <return_stmt>
                       | <expr_stmt>

<var_decl>          ::= "var" IDENT ":" <type> "=" <expr>
<assign_stmt>       ::= IDENT "=" <expr>
<if_stmt>            ::= "if" <expr> <block> [ "else" <block> ]
<while_stmt>        ::= "while" <expr> <block>
<run_stmt>          ::= "run" IDENT
<return_stmt>       ::= "return" <expr>
<expr_stmt>         ::= <call_expr>

<expr>              ::= <or_expr>
<or_expr>           ::= <and_expr> { "||" <and_expr> }
<and_expr>          ::= <eq_expr> { "&&" <eq_expr> }
<eq_expr>           ::= <cmp_expr> { ( "==" | "!=" ) <cmp_expr> }
```

```

<cmp_expr>      ::= <add_expr> { ( "<" | "<=" | ">" | ">=" ) <add_expr> }
<add_expr>      ::= <mul_expr> { ( "+" | "-" ) <mul_expr> }
<mul_expr>      ::= <unary_expr> { ( "*" | "/" ) <unary_expr> }
<unary_expr>    ::= ( "-" | "!" ) <unary_expr> | <primary>
<primary>       ::= INT_LIT
                  | FLOAT_LIT
                  | BOOL_LIT
                  | <call_expr>
                  | <field_expr>
                  | IDENT
                  | "(" <expr> ")"

<call_expr>     ::= IDENT "(" [ <args> ] ")"
<args>          ::= <expr> { "," <expr> }
<field_expr>    ::= IDENT "." <field_name>
<field_name>    ::= IDENT | "timeout" | "retries" | "parallel"

```

Listing 2: WorkflowScript EBNF grammar

3.2 Ambiguity Notes

3.2.1 Dangling Else

The `<if_stmt>` rule with an optional `else` is potentially ambiguous when `if` statements are nested. WorkflowScript resolves this with the nearest-if rule: an `else` clause always binds to the innermost unmatched `if`. This is the same convention used by C, Go, and Java. The recursive-descent parser implements it naturally: after parsing the `then` block, it looks for `else` before returning, so the `else` is consumed by the current (innermost) `if` invocation.

3.2.2 Operator Precedence

Precedence is encoded directly into the grammar stratification rather than via a separate table. From lowest to highest binding strength:

Level	Operators
Lowest	 && == != < <= > >= + - * /
Highest	unary - !

Table 2: Operator precedence levels (lowest to highest)

Because each grammar level calls the level above it, there is no ambiguity about how an expression like `a + b * c == d || e` parses — the structure is determined by the grammar alone.

3.2.3 Field Access on Task-Field Keywords

The identifiers `timeout`, `retries`, and `parallel` are lexed as keyword tokens, not as `IDENT`. A naive grammar would make `t.timeout` unparseable because `.` would be followed by a keyword rather than an

IDENT. The `<field_name>` non-terminal explicitly allows all three task-field keywords on the right-hand side of `.`, resolving this.

3.2.4 Expression Statement Restriction

Only call expressions are permitted as standalone statements (`<expr_stmt> ::= <call_expr>`). A bare variable reference used as a statement (e.g., writing `x` on its own line) is rejected with a parser error. This eliminates the syntactic ambiguity between a no-op expression statement and a misspelled assignment, and prevents silent no-op bugs.

3.2.5 Assignment vs. Expression

Assignment is a statement (`<assign_stmt>`), not an expression. The parser distinguishes IDENT `"=`" (assignment) from IDENT `"("` (call) by one token of lookahead. This prevents the classic `if x = 5` bug and removes the need for a separate assignment-expression production.

4 Names, Binding, Scope, and Lifetime

4.1 Legal Identifiers

A name must match the regular expression `[a-zA-Z][a-zA-Z0-9_]*`: it begins with a letter or underscore and continues with any mix of letters, digits, and underscores. There is no length limit. Names are case-sensitive: `Build` and `build` are distinct identifiers. Any name that matches a keyword listed in Section 3 is reserved and cannot be used as a user-defined identifier.

4.2 Bindings: Compile Time vs. Run Time

Binding	Time
Variable type (declared in <code>var</code>)	Compile time — explicit type annotation
Function parameter types and return type	Compile time — explicit annotations
Task field types (<code>timeout: int</code> , <code>retries: int</code> , <code>parallel: bool</code>)	Compile time — fixed by language definition
Stage-to-task resolution in pipelines	Compile time — stage IDENT must name a declared task
Variable value	Run time
Function body execution	Run time
Pipeline execution (on <code>run</code>)	Run time

Table 3: Binding times in WorkflowScript

The early binding of types and stage references is the primary mechanism by which WorkflowScript achieves reliability. A program that passes the type checker is guaranteed to have no unresolved task references at runtime, and no type mismatches in arithmetic or comparisons.

4.3 Scoping

WorkflowScript uses **static (lexical) scoping**. The scope of a name is determined by its textual position in the source. A name is visible from its declaration point to the end of the enclosing block. Inner blocks may shadow outer names.

There are three kinds of scope:

1. **Program scope** — the outermost scope. Task declarations, pipeline declarations, top-level `var` declarations, and top-level statements all live here. Every name declared at program scope is visible throughout the entire program after its declaration point.
2. **Function scope** — each `fun` body opens a new scope. Parameters are bound at the function entry point and are visible throughout the function body. `var` declarations inside the body extend their scope to the closing `}` of the function.
3. **Block scope** — `if` bodies, `while` bodies, and `on_failure` bodies each open a nested scope. A `var` declared inside one of these blocks is not visible outside it.

4.3.1 Why Static Scoping?

A developer can determine which variable a name refers to by reading the source alone, without executing the program. Dynamic scoping would make name resolution call-order-dependent. In a pipeline context this is especially dangerous: a function called from two different `on_failure` handlers could see different variable bindings depending on which handler invoked it. This makes the same source code mean different things in different call contexts — precisely the kind of unreliability WorkflowScript is designed to prevent.

4.3.2 What Would Break Under Dynamic Scoping?

Consider a function `fun check(t: task) -> bool` that reads a top-level variable `var threshold: int`. With static scoping, `threshold` always resolves to the top-level declaration, regardless of where `check` is called. With dynamic scoping, if a caller had a local variable named `threshold`, the function would silently read the caller’s value instead. Pipeline authors would have to track all active variable names at every call site — an unreasonable cognitive burden.

4.4 Lifetime

Entity	Lifetime	Rationale
<code>task</code> declarations	Static — entire program execution	Stage references must remain valid whenever a pipeline runs
<code>pipeline</code> declarations	Static — entire program execution	Pipelines may be invoked anywhere in the program
Top-level <code>var</code> declarations	Static — entire program execution	Top-level variables function as global pipeline configuration
Function parameters	Stack-dynamic — created on call, destroyed on return	Standard; no reason to outlive the invocation
Local <code>var</code> inside a block	Stack-dynamic — created on block entry, destroyed on exit	Standard block-scoped allocation

Table 4: Lifetime of entities in WorkflowScript

4.4.1 Why Static Lifetime for Tasks?

A pipeline’s `stage` declarations are resolved at compile time, but the pipeline may be executed (via `run`) at any point during program execution. If tasks were stack-dynamic, a task declared inside a function could be destroyed before a pipeline referencing it runs. Making task lifetime static ensures that any `stage` reference that passes the type checker is also safe at runtime — the referenced task will always exist.

4.4.2 Why Stack-Dynamic for Local Variables?

Local variables need to hold different values across different function calls and loop iterations. Stack-dynamic allocation achieves this naturally: each call to a function creates a fresh binding for each parameter and local variable. Explicit-heap-dynamic allocation would add unnecessary complexity (requiring manual deallocation or a garbage collector) for values that are structurally bounded by the block they live in.